



Badanie przejrzystości wody

System monitorowania TDS z Arduino Uno

Jakub Dostojewski

Klasa 3F

15 kwietnia 2026

Opis techniczny

Urządzenie wykorzystuje czujnik analogowy TDS oraz DHT11 do precyzyjnego pomiaru zanieczyszczenia cieczy z kompensacją temperaturą.

Spis treści

1	Wstęp	3
1.1	Spis sprzętu 	3
1.2	Podstawy teoretyczne 	3
1.3	Wykorzystane oprogramowanie 	3
2	Zasada działania systemu 	4
3	Kod źródłowy programu 	4
4	Schematy 	7
4.1	Schemat uproszczony (Fritzing)	7
4.2	Schemat rzeczywisty (KiCad)	8

1 Wstęp

1.1 Spis sprzętu

-  Arduino Uno rev. 3 — 1 szt.
-  Czujnik TDS — 1 szt.
-  Czujnik DHT11 — 1 szt.
-  Wyświetlacz LCD 16x2 — 1 szt.
-  Dioda LED (czerwona) — 1 szt.
-  Rezystory 220Ω — 2 szt.
-  Płytki stykowe i przewody

1.2 Podstawy teoretyczne

Czym jest TDS?

TDS (*Total Dissolved Solids*) mierzy całkowitą ilość substancji rozpuszczonych w wodzie. Pomiar odbywa się poprzez badanie **przewodności elektrycznej**. Wartość podawana jest w jednostkach **ppm** (*parts per million*).

1.3 Wykorzystane oprogramowanie

Środowisko programistyczne

- Arduino IDE 2.3.8 dla systemu Arch Linux™ – tworzenie i wgrywanie programu do mikrokontrolera
- Fritzing 1.0.6 dla systemu Microsoft Windows 11 – wykonanie schematu połączeń na płytce stykowej
- KiCad 10.0 dla systemu Arch Linux™ – profesjonalny schemat elektryczny
- TeXstudio 4.9.3 dla systemu Arch Linux™ – przygotowanie dokumentacji projektu w $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$

2 Zasada działania systemu

Opis działania

Układ mierzy przewodność elektryczną cieczy za pomocą czujnika TDS. Na podstawie odczytanego napięcia obliczana jest wartość ppm. Temperatura odczytywana z czujnika DHT11 służy do kompensacji błędu pomiarowego (jest to temperatura otoczenia - do obliczeń w stopniach Celsjusza, natomiast ta wyświetlana na ekranie LCD jest przeliczona na stopnie Kelvina). Wyniki są wyświetlane na ekranie LCD, a przekroczenie ustalonego progu zanieczyszczenia powoduje zapalenie się diody LED.

3 Kod źródłowy programu

Implementacja w języku C++ z wykorzystaniem filtracji medianowej

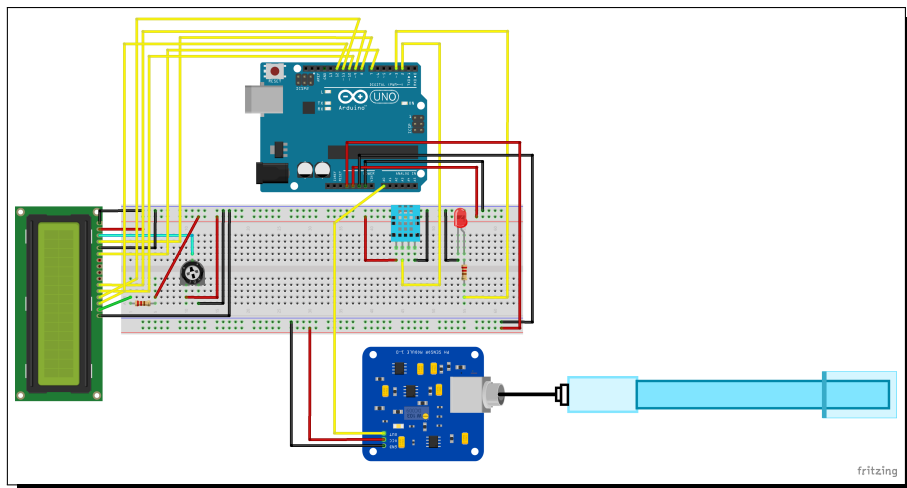
```
1 // Libraries
2 #include <LiquidCrystal.h>
3 #include <dht.h>
4
5 // Constant variables and pins
6 #define TdsSensorPin A0
7 #define dhtPin 2
8 #define ledPin 3
9 #define VREF 5.0 // Reference voltage of ADC
10 #define SCOUNT 30 // Number of samples for median filter
11
12 int analogBuffer[SCOUNT]; // Buffer for raw TDS samples
13 int analogBufferTemp[SCOUNT]; // Temporary buffer for sorting
14 int analogBufferIndex = 0, copyIndex = 0, check;
15
16 float averageVoltage = 0, tdsValue = 0, Temperature, temperatureKelvin;
17
18 // Members of classes: lcd for LiquidCrystal (display) and DHT for dht
19 // class (sensor)
20 LiquidCrystal lcd(7, 8, 9, 10, 11, 12);
21 dht DHT;
22
23 // Setting serial port, lcd display and pins
24 void setup() {
25     lcd.begin(16, 2);
26     pinMode(TdsSensorPin, INPUT);
27     pinMode(ledPin, OUTPUT);
28 }
```

```
27 pinMode(dhtPin, INPUT);
28 Serial.begin(9600);
29 }
30
31 void loop() {
32     // Reading temperature from DHT11 and calculating Kelvin scale
33     static unsigned long analogSampleTimepoint = millis();
34     check = DHT.read11(dhtPin);
35     if(check == 0) Temperature = DHT.temperature;
36     temperatureKelvin = Temperature + 273.15;
37
38     // Sampling TDS sensor every 40ms and storing in circular buffer
39     if (millis() - analogSampleTimepoint > 400) {
40         analogSampleTimepoint = millis();
41         analogBuffer[analogBufferIndex] = analogRead(TdsSensorPin);
42         analogBufferIndex++;
43         if (analogBufferIndex == SCOUNT) analogBufferIndex = 0;
44     }
45
46     static unsigned long printTimepoint = millis();
47
48     // Processing and displaying data every 800ms
49     if (millis() - printTimepoint > 800) {
50         printTimepoint = millis();
51
52         // Copying buffer for filtering (to keep original data intact)
53         for (copyIndex = 0; copyIndex < SCOUNT; copyIndex++)
54             ↪ analogBufferTemp[copyIndex] = analogBuffer[copyIndex];
55
56         // Getting median voltage and converting to real voltage value
57         averageVoltage = getMedianNum(analogBufferTemp, SCOUNT) * (float)VREF
58             ↪ / 1024.0;
59
60         // Temperature compensation (standard reference 25°C)
61         float compensationCoefficient = 1.0 + 0.02 * (Temperature - 25.0);
62         float compensationVoltage = averageVoltage / compensationCoefficient;
63
64         // Converting voltage to TDS value (ppm) using polynomial equation
65         tdsValue = (133.42 * pow(compensationVoltage, 3) - 255.86 *
66             ↪ pow(compensationVoltage, 2) + 857.39 * compensationVoltage) * 0.5;
67
68         // High TDS alarm - LED indicator
69         if(tdsValue >= 300) digitalWrite(ledPin, HIGH);
70         else digitalWrite(ledPin, LOW);
71
72         // Refreshing LCD with TDS results and temperature in Kelvin
73         lcd.clear();
74         lcd.setCursor(0, 0);
75         lcd.print("TDS: ");
```

```
73     lcd.print(tdsValue, 0);
74     lcd.print("ppm");
75
76     lcd.setCursor(0, 1);
77     lcd.print("T: ");
78     lcd.print(temperatureKelvin, 0);
79     lcd.print("K");
80 }
81 }
82
83 // Median filter function: sorts the array and returns the middle value
84 int getMedianNum(int bArray[], int iFilterLen) {
85     int bTab[iFilterLen];
86     for (byte i = 0; i < iFilterLen; i++) bTab[i] = bArray[i];
87
88     int i, j, bTemp;
89
90     // Simple Bubble Sort algorithm
91     for (j = 0; j < iFilterLen - 1; j++) {
92         for (i = 0; i < iFilterLen - j - 1; i++) {
93             if (bTab[i] > bTab[i + 1]) {
94                 bTemp = bTab[i];
95                 bTab[i] = bTab[i + 1];
96                 bTab[i + 1] = bTemp;
97             }
98         }
99     }
100
101     // Returning median based on odd/even length
102     if ((iFilterLen & 1) > 0) bTemp = bTab[(iFilterLen - 1) / 2];
103     else bTemp = (bTab[iFilterLen / 2] + bTab[iFilterLen / 2 - 1]) / 2;
104
105     return bTemp;
106 }
```

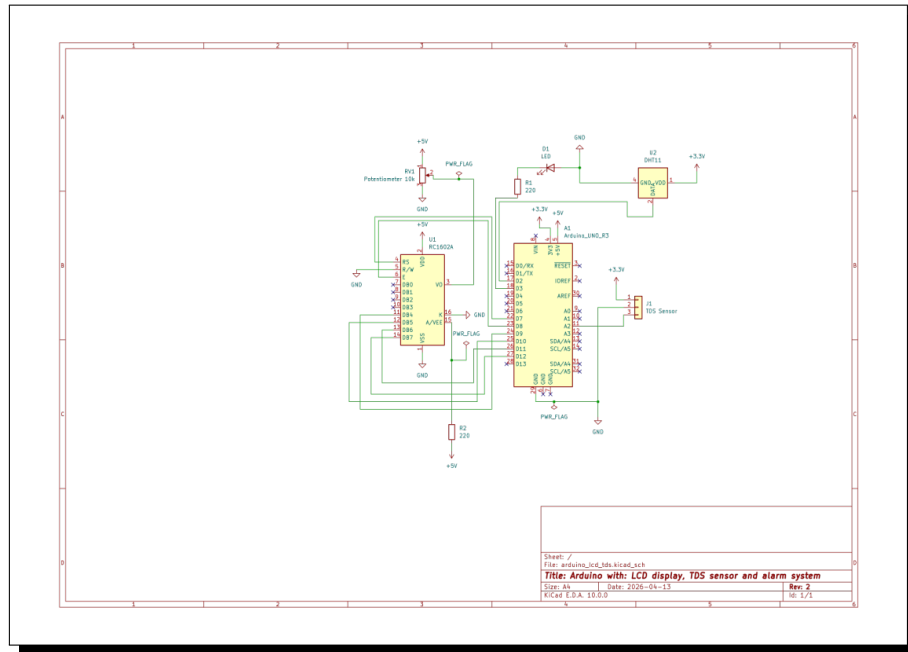
4 Schematy

4.1 Schemat uproszczony (Fritzing)



Rysunek 1: Wizualizacja połączeń na płytce stykowej.

4.2 Schemat rzeczywisty (KiCad)



Rysunek 2: Schemat elektryczny układu pomiarowego.